

Common Information:

For each value of n , we will construct a $n \times n$ or $n \times 2n$ matrix first based on the information. The first loop will iterate through the columns and the second loop inside the first loop will be the rows..

Type 1: Left Half Pyramids

$n=3$

* (1,1)	(1,2)	(1,3)
*(2,1)	*(2,2)	(2,3)
*(3,1)	*(3,2)	*(3,3)

For each column, the star is printed until the value of row is lesser than the value of the column.
If i = column , and j = row, print star until $j \leq i$

Code:

```
#include <stdio.h>
```

```
int main() {  
    int rows = 5;  
  
    // This loop for traverse  
    // pyramid from top to bottom  
    for (int i = 0; i < rows; i++) {  
  
        // Inner loop for printing  
        // character in each rows  
        for (int j = 0; j <= i; j++) {  
            printf("* ");  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

Variants of the left half pyramid:

Code	Output
<pre>for(int i = 1; i <= n; i++){ for(int j = 1; j <= i; j++){ printf("* "); } printf("\n"); }</pre>	<pre>* * * * * * * * * * * * * * *</pre>
<pre>for(int i = 1; i <= n; i++){ for(int j = 1; j <= i; j++){ printf("%d ", j); } printf("\n"); }</pre>	<pre>1 1 2 1 2 3 1 2 3 4</pre>
<pre>for(int i = 1; i <= n; i++){ for(int j = 1; j <= i; j++){ printf("%d ", i); } printf("\n"); }</pre>	<pre>1 2 2 3 3 3 4 4 4 4</pre>
<pre>for(int i = 1; i <= n; i++){ for(int j = 1; j <= i; j++){ printf("%c ", 'A' + j - 1); } printf("\n"); }</pre>	<pre>A A B A B C A B C D</pre>
<pre>for(int i = 1; i <= n; i++){ for(int j = i; j >= 1; j--){ printf("%d ", j); } printf("\n"); }</pre>	<pre>1 2 1 3 2 1 4 3 2 1</pre>

<pre> int k = 1; for(int i = 1; i <= n; i++){ for(int j = 1; j <= i; j++){ printf("%d ", k++); } printf("\n"); } </pre>	<pre> 1 2 3 4 5 6 7 8 9 10 </pre>
<pre> for(int i = 1; i <= n; i++){ for(int j = 1; j <= i; j++){ printf("%d ", (i + j) % 2); } printf("\n"); } </pre>	<pre> 1 0 1 1 0 1 0 1 0 1 </pre>

Right Half Pyramid:

Code

```

#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++) {
        // Print spaces
        for(j = 1; j <= n - i; j++) {
            printf(" ");
        }

        // Print stars
        for(j = 1; j <= i; j++) {
            printf("*");
        }
        // Move to the next line
        printf("\n");
    }
    return 0;
}

```

```
}
```

Full Pyramid Logic:

Use 2 loops ->

Print (n - i) spaces

Print (2*i - 1) stars

Move to next line

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, j;
```

```
    printf("Enter number of rows: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 1; i <= n; i++) {
```

```
        // Step 1: Print spaces
```

```
        for(j = 1; j <= n - i; j++) {
```

```
            printf(" ");
```

```
        }
```

```
        // Step 2: Print stars
```

```
        for(j = 1; j <= 2 * i - 1; j++) {
```

```
            printf("*");
```

```
        }
```

```
        // Step 3: Move to next line
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Inverted Pyramid:

The logic is just the exact opposite to a normal full pyramid. Only change is printing in the number of spaces in each row.

Print $(i - 1)$ spaces #instead of $n-i$ spaces

Print $(2(n - i) + 1)$ * stars

Move to next line

Code

```
#include <stdio.h>
```

```
int main() {  
    int n, i, j;
```

```
    printf("Enter number of rows: ");  
    scanf("%d", &n);
```

```
    for(i = 1; i <= n; i++) {
```

```
        // Step 1: Print spaces  
        for(j = 1; j <= i - 1; j++) {  
            printf(" ");  
        }
```

```
        // Step 2: Print stars  
        for(j = 1; j <= 2 * (n - i) + 1; j++) {  
            printf("*");  
        }
```

```
        // Step 3: Move to next line  
        printf("\n");  
    }
```

```
    return 0;  
}
```

Diamond Pattern:

Logic:

A diamond = Full pyramid (top half) and Inverted full pyramid (bottom half)

For row i from 1 to n:

Print (n - i) spaces

Print (2*i - 1) stars

For row i from n-1 down to 1:

Print (n - i) spaces

Print (2*i - 1) stars

Code:

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++) { // Top half (full pyramid)
        for(j = 1; j <= n - i; j++) { // Print spaces
            printf(" ");
        }
        for(j = 1; j <= 2 * i - 1; j++) { // Print stars
            printf("*");
        }
        printf("\n");
    }

    for(i = n - 1; i >= 1; i--) { // Bottom half (inverted pyramid)
        for(j = 1; j <= n - i; j++) { // Print spaces
            printf(" ");
        }
        for(j = 1; j <= 2 * i - 1; j++) { // Print stars
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

Hollow Pattern:

Logic:

If it is the first row or last row → print all stars, Otherwise (middle rows):

- Print * (left border)

- Print spaces (n - 2) times

- Print * (right border)

Code:

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter size of square: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            if(i == 1 || i == n || j == 1 || j == n)
                printf("*");
            else
                printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

Cross Pattern:

Logic:

We print a star in two diagonal positions:

- Left diagonal → row index equals column index, $i == j$

- Right diagonal → row + column = n + 1, $i + j == n + 1$

Code:

```
for(i = 1; i <= n; i++) {
    for(j = 1; j <= n; j++) {
        if(i == j || i + j == n + 1)
            printf("*");
        else printf(" ");
    }
```